CHULA INTERNATIONAL SCHOOL OF ΣNGINEERING
The ingenuity of CHULA ΣNGINEERING

*Industrial Robot Project Report 2147308*

# Project: Pick up box

*By*

**Thanakorn Sappakit, Nunnapat Anupongongarch, Tibet Buramarn, Thitaya Divari, Tinapat Limisila, Chanapa Anuwatvimol, Tunyaluck Liengkorsakul**

Robotics & AI Program, International School of Engineering, Chulalongkorn University
254 Phayathai Rd, Wang Mai, Pathum Wan, Bangkok, Thailand 10330

## Abstract

## Keyword

# Contents

# 1 Introduction

## 2 Specifications

In this project, a robotic arm is tasked with detecting and picking up one of two boxes as they move slowly along a conveyor belt. The system uses a camera to detect the presence of a box, after which the robot automatically grabs it. The robotic arm starts at the midpoint of the conveyor belt, with its base coordinates defined in Table 1.

Table 1: Project Specifications

| **Specifications** | **Value** |
|---|---|
| Xbase | 116 mm |
| Ybase | -300 mm |
| Zbase | 80 mm |
| Rxbase | 2.233 rad |
| Rybase | 2.257 rad |
| Rzbase | -0.039 rad |

After the first box passes the midpoint, a second box is placed at the far right end of the conveyor. This box will also be detected, grasped, and lifted by the robot to at least the height of its initial position. The second box is positioned randomly on the belt but aligned along the conveyor belt axis, with an orientation deviation of no more than $\pm 10^{\circ}$.

## 3 Methodology

This project integrates a UR3 robotic arm with a vision system and conveyor belt for autonomous pick-and-place operations. The system uses TCP/IP communication to transmit coordinates and commands between the robot, gripper, and conveyor. A Python-based control loop manages the synchronization between the vision system, robot, and conveyor, ensuring precise box detection and positioning. The vision system detects the position of boxes, and the robot arm moves accordingly to pick and place the objects.

The software implementation involves a custom Robot class for controlling the UR3 arm, which simplifies movement commands by using absolute pose instructions. The system accounts for delays, such as camera-to-computer latency, by predicting future box positions. Extensive testing was conducted to ensure accurate box detection, robot arm movements, and gripper functionality. This methodology enables reliable and efficient robotic operations within a dynamic environment.

# 4 TCP Communication

A widely adopted communication method for interfacing with Universal Robots is the use of TCP/IP socket connections over Ethernet. This approach is favored for its simplicity and reliability. Universal Robots support several built-in Remote Control Servers including the Primary, Secondary, Real-time, and Dashboard servers all of which operate using the TCP/IP protocol. These servers are designed to receive and process remote commands from client applications running on external devices or other robotic systems, enabling seamless remote operation and integration.

The integration of TCP/IP socket communication in the UR3 robotic system provides a remote control and data exchange across all components including robot arm, gripper, vision system, and conveyor. Leveraging Python's socket programming capabilities, the system establishes reliable channels for real-time command execution and sensor feedback, ensuring high accuracy and repeatability in industrial pick-and-place applications.

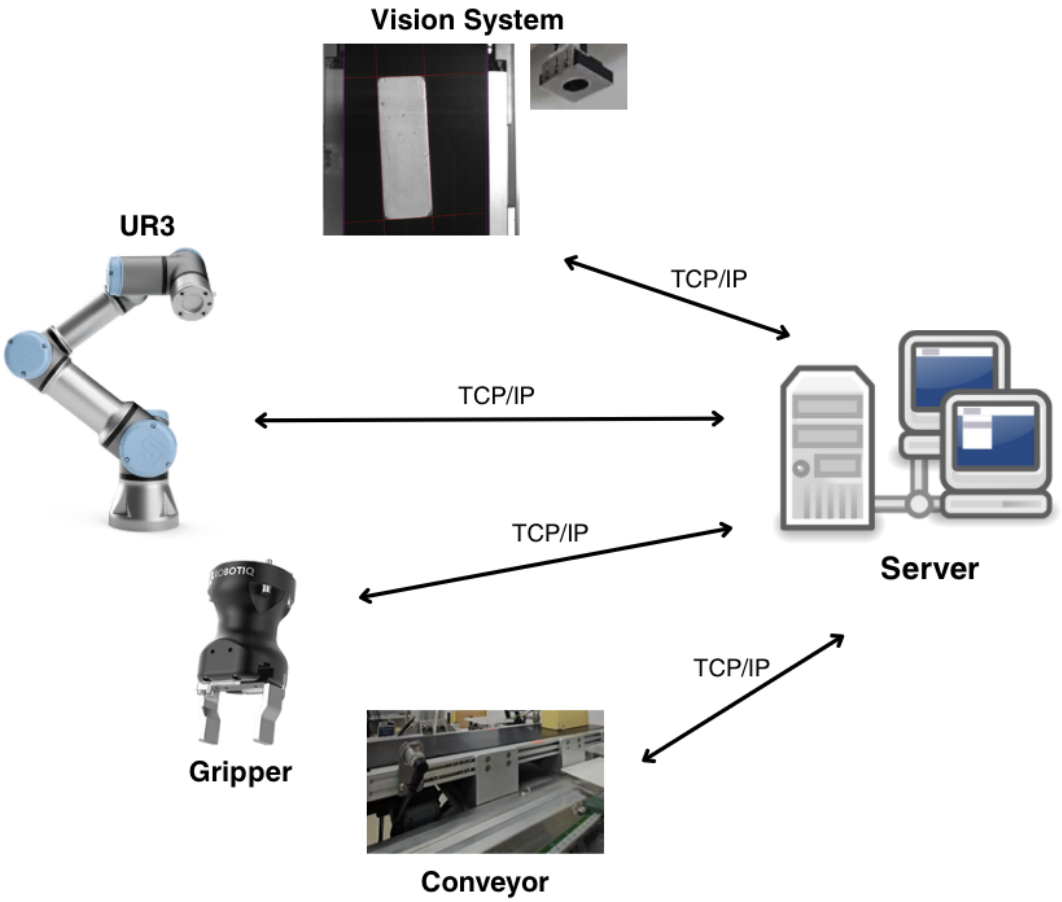


Figure 1: TCP Communication between Components

By adopting this architecture, developers can interface with the UR3 and external vision systems and dynamic task coordination. The use of TCP/IP not only enhances communication reliability but also simplifies integration with other industrial devices, making it ideal for modular, remote-controlled robotic systems.

# 5 Vision Builder

# 6 UR3 Arm

In this project, we utilize the UR3 robotic arm, a compact, six-degree-of-freedom (6-DOF) collaborative robot known for its precision, flexibility, and ease of integration. The UR3 offers 360° rotation on all wrist joints and unlimited rotation on the end joint, providing high flexibility in confined workspaces. We employ the UR3 to perform reliable and safe object grasping in dynamic environments. By leveraging its programmable joint movements and end-effector control, the UR3 can accurately approach, align with, and grasp a variety of target objects. It is controlled via a custom robot class that simplifies programming through high-level commands, which will be explained in more detail in the integration section.

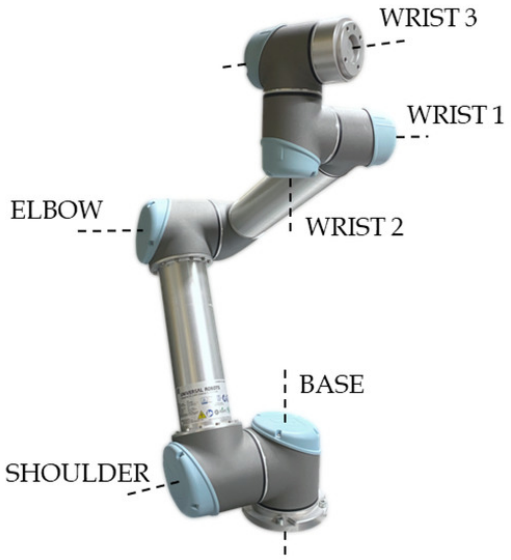


Figure 2: *Labeled structure of the UR3 robotic arm, showing the six rotational joints: Base, Shoulder, Elbow, Wrist 1, Wrist 2, and Wrist 3. [1]*

Figure 2 illustrates the UR3 robotic arm, which comprises six revolute joints: Base, Shoulder, Elbow, Wrist 1, Wrist 2, and Wrist 3. The corresponding joint configurations and their operating ranges, each allowing a rotation of ±360°, are detailed in Table 2.

Table 2: *UR3 Robotic Arm Joint Movements and Corresponding Operating Ranges.*

| **Robotic Arm Axis Movement** | **Operating Range** |
|---|---|
| Base | ±360° |
| Shoulder | ±360° |
| Elbow | ±360° |
| Wrist 1 | ±360° |
| Wrist 2 | ±360° |
| Wrist 3 | ±360° |

# 7 Integration

## 7.1 Software Architecture and Communications Framework

The software integration for the system implements a communication framework connecting three primary components: the vision system, the UR3 robotic arm, and the conveyor belt control system. This integration enables real-time detection of boxes moving along the conveyor and robotic arm movements to retrieve the targets.

## 7.2 TCP Communication Architecture

The system utilizes Transmission Control Protocol (TCP) socket connections to establish reliable communication channels between different components. This architecture ensures correctness within the transmitted data between different systems:

- A TCP connection to the vision system (IP: 10.10.1.10, Port: 2024) receives continuous coordinate data of detected boxes
- A TCP connection to the UR3 robot controller (IP: 10.10.0.14, Port: 30003) sends movement commands in real-time
- A TCP connection to the gripper controller (IP: 10.10.0.14, Port: 63352) controls opening and closing actions
- A TCP connection to a local conveyor controller (IP: 10.10.0.98, Port: 2002) manages conveyer belt operations

The choice of TCP over alternatives like UDP ensures reliable and correct data delivery with error checking, which is critical for the precision required in robotic picking applications.

## 7.3 Vision System Integration

1. **Data Reception and Parsing:** The system receives coordinate strings from the vision system in a custom format with sides delimited by "/" characters and coordinates by commas. This data provides the position of the box's left, right, top, and bottom edges.
2. **Motion Analysis:** The software implements a simple but effective motion tracking algorithm that:
    - Records the initial y-position when a box is first detected
    - Calculates the change in position across frames to determine velocity
    - Converts pixel measurements to physical distances using a calibration constant (0.2 meters across the frame)
3. **Position Estimation:** The system calculates where to position the gripper by:
    - Determining the center point between the left and right edges of the box
    - Converting pixel coordinates to physical distances using a calibration constant
    - Applying offsets to account for the camera's perspective

This approach provides sufficient information to predict where the box will be when the robot reaches it, compensating for the inherent delays in the system.

## 7.4 UR3 Robot Arm Control

The control of the UR3 robotic arm is implemented through a custom Robot class that abstracts the complexities of robot programming:

1. **Position Control:** The robot is controlled through absolute pose commands (movel) that specify the end-effector's position and orientation in Cartesian space. This approach simplifies the trajectory planning by delegating inverse kinematics calculations to the UR3 controller.
2. **Home Position:** A predefined home position is established as a reference point for all movements, with additional movements defined relative to this position using the *move_wrt_home* method.

3. **Timing Management:** The system accounts for various delays in the control loop:
    - Camera-to-computer delay (0.25 seconds)
    - Robot movement delay (0.35 seconds)
    - These delays are factored into the position calculations to compensate for the conveyor belt's continuous motion
4. **Calibration Parameters:** The code incorporates calibration constants to translate between pixel coordinates and physical distances, ensuring accurate positioning of the robot arm.

## 7.5 Synchronized Operation Logic

The integration synchronizes all components through a main control loop that follows a sequential process:

1. **Initialization:** The system initializes all connections and sets the robot to its home position.
2. **Conveyor Activation:** The conveyor belt is activated at a specified speed (20 units).
3. **Control Loop:** The control loop continuously:
    - Receives vision data
    - Processes box positions to detect new boxes and track movement
    - Calculates box velocity and predicts future positions
    - Prepares the gripper based on detected box size
    - Executes picking maneuvers at calculated intercept points
    - Returns to home position after successful picks

The loop implements timing constraints to ensure that operations occur within the narrow window available for successful picking. The system employs a predictive approach where the robot is commanded to move to where the box will be, rather than where it currently is, accounting for system delays.

## 7.6 Performance Considerations

Several performance optimizations are evident in the implementation:

- **Frame Rate Monitoring:** The system calculates frames per second (fps) to maintain awareness of the vision system's processing speed, which affects motion calculations.
- **Adaptive Positioning:** The grabbing position is dynamically adjusted based on the calculated speed of the conveyor belt and the system's processing delays.
- **Error Bounds:** The code includes constraints to keep motion parameters within safe limits, preventing extreme movements that could damage the system.

The integration balances complexity with reliability, creating a system capable of precisely timing robotic movements to intercept moving objects on a conveyor belt—a challenging task that requires precise coordination between vision, robot control, and motion prediction.

# 8 Conclusion

# References

[1] A. Raviola, R. Guida, A. C. Bertolino, A. De Martin, S. Mauro, and M. Sorli, "A comprehensive multibody model of a collaborative robot to support model-based health management," *Robotics*, vol. 12, no. 3, 2023. [Online]. Available: https://www.mdpi.com/2218-6581/12/3/71